

NUST CHIP DESIGN CENTRE

Physical Design

Lab # 2
Synthesis of Single Cycle Processor

Name	***Sidra Naveed***
Instructor	*Ms. Hira Sohail*
Date	*6th October 2025*

Introduction to Synthesis:

Synthesis is the process of converting high level hardware description language written in RTL into gate level netlist. This netlist is made of standard cells which are provided through technology library. The objective of the synthesis to create a circuit which meets the functional correctness, timing, area and power requirements for the design.

In synthesis, the tools which we are using as Cadenece genus tool, maps RTL modules like registers counter and logic combinations onto logic gates, flip flops and multiplexers available in the liberty file.

Timing constraints, specified in the Synopsys Design Constraints (SDC) file, guide the tool to meet setup, hold, and propagation delay requirements.

Main Files:

Constraint File (.sdc)

The **constraint file**, which is called Synopsys Design Constraints (SDC) format. It provides the synthesis tool with the timing and design requirements it must follow. It defines important clock properties such as period, duty cycle, and rise/fall times. It also checks violations of setup and hold margins through clock uncertainty. It specifies how inputs and outputs should be timed by setting input delays, output delays, and input transition times relative to the clock.

By applying these constraints, the tool ensures the generated gate-level netlist behaves correctly under real hardware conditions, preventing setup/hold violations.

Script File (.tcl)

The **synthesis script**, written in Tcl language, automates the entire synthesis flow in tools like Cadence Genus. It starts by setting the design name, loading the standard cell libraries, and reading in the RTL files. Next, it applies the constraints from the .sdc file, elaborates the design, and runs the synthesis process—covering generic mapping, technology mapping, and optimization. At the end, it produces the mapped netlist, an SDF delay file, an updated constraint file, and detailed reports on timing, area, power, and gate count. Using a Tcl script guarantees repeatability, automation, and consistency, which are critical for efficient design iterations.

RTL File (.v)

The **RTL file**, written in Verilog, describes how the design works at the register-transfer level. It defines the behavior of modules like counters, state machines, or arithmetic circuits, and serves as the starting point for synthesis. The synthesis tool takes this high-level description and translates it into a gate-level netlist using cells from the target library. Since any bug in the RTL will carry forward into the synthesized hardware, the RTL must be thoroughly verified beforehand. In short, the Verilog RTL acts as the blueprint of the digital system being implemented.

Library File (.lib)

The **library file** contains the characterization data for the standard cells provided by a specific technology node (for example, 65 nm). Each .lib entry describes the behavior of a cell—its logic function, propagation delay, setup and hold times, power usage, and capacitance. Common cells include inverters, NAND gates, and flip-flops. During synthesis, the RTL design is mapped onto these cells while ensuring that all timing and power constraints are satisfied. Without this library, the synthesis tool cannot translate RTL into a real

hardware implementation. In this lab, the `tcbn65lptc.lib` library was used to map the counter design onto 65 nm standard cells.

Genus Tool:

Genus is the tool developed by cadence used to synthesis the rtl design. It helps till the frontend of the physical design.

Commands:

csh

This command starts the **C shell (Csh)**. Many EDA tools (like Cadence) are configured to run in csh because their setup scripts are written for it.

source cshrc

This command **loads the environment setup** from the `.cshrc` file. The following screenshot is taken from cshrc file given.

```
setenv CDS_LIC_FILE $LM_LICENSE_FILE
setenv CDS_LIC_QA_TesT "./qaLog.txt"
setenv LANG "en_US"
setenv QRC_EXTRACTION_ENABLE ""
setenv CDS_LIC_ONLY "1"
setenv CDS_AUTO_64BIT "ALL"
#setenv LD_PRELOAD "/usr/lib64/libssl.so.1.0.2k:/usr/lib64/libcrypto.so.1.0.2k"

setenv CDSHOME /home/cc/mnt/IC231
setenv INNOVUSHOME /home/cc/mnt/INNOVUS211/tools.lnx86

setenv XRUN_HOME /home/cc/mnt/XCELIUM2309

setenv QRC_HOME /home/cc/mnt/QUANTUS231

setenv SPECTRE_HOME /home/cc/mnt/SPECTRE231
```

Figure 1:cshrc file

This script prepares the environment by setting license details, tool paths, and runtime options, so that when you type genus -f script.tcl, the system immediately finds and runs the correct Cadence tool.

genus -f script.tcl

After system configuration, this command is used to start the cadence tool genus. It runs with the help of script.tcl. The file script.tcl works as the combination of different commands to run.

```
[cc@ncdc-0053 Sidra_PD]$ csh
[cc@ncdc-0053 Sidra_PD]$ source ~/cshrc
**************************************************
******        Welcome to Cadence Tools         *******
**************************************************

[cc@ncdc-0053 Sidra_PD]$ genus -f DFT_script.tcl
2025/10/01 15:29:57 WARNING This OS does not appear to be a Cadence supported Linux configuration.
2025/10/01 15:29:57 For more info, please run CheckSysConf in <cdsRoot/tools.lnx86/bin/checkSysConf <productId>
TMPDIR is being set to /tmp/genus_temp_22667_ncdc-0053_cc_tdn8GB
Cadence Genus(TM) Synthesis Solution.
Copyright 2023 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks and Genus is a trademark
of Cadence Design Systems, Inc. in the United States and other countries.
```

Script.tcl:

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


##############################################################################################################
## Design Setup
##############################################################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       counter

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

Figure 2:script.tcl

Explanation of script.tcl:

The **script.tcl** file automates the entire synthesis flow in Cadence Genus. Here is the main flow of this script.

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}
```

It starts by setting up the environment—loading the technology libraries (.lib) and defining the top-level design module.

```
# Read HDL source files (Verilog 2001 in this case)
# IMPORTANT: Update the path if your Verilog source is located elsewhere
read_hdl -v2001 {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/RTL/counter.v}

# Elaborate the top-level module
elaborate $DESIGN

# Check design for unresolved references (ensures all modules are linked correctly)
check_design -unresolved
```

Next, it reads in the RTL source file and elaborates it to catch any missing references.

```
# Read constraints file (SDC format)
# Contains clock definitions, input/output delays, etc.
read_sdc {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/constraints/counter.sdc}
```

The design constraints from the .sdc file are then applied.

```
# Step 1: Generic Synthesis (technology-independent RTL → generic netlist)
set_db / .syn_generic_effort $GEN_EFF
syn_generic

# Step 2: Technology Mapping (map generic netlist to standard cells from library)
set_db / .syn_map_effort $MAP_OPT_EFF
syn_map

# Step 3: Netlist Optimization (timing, area, power optimizations)
set_db / .syn_opt_effort $MAP_OPT_EFF
syn_opt
```

Once the setup is complete, the script runs the synthesis in three stages: generic synthesis, technology mapping, and optimization for timing, power, and area.

```
# Write synthesized Verilog netlist (mapped to library cells)
write_hdl $DESIGN -mapped   >> ./Netlist/${DESIGN}_map.v

# Write constraints file for backend PnR (to be used in Innovus)
write_sdc $DESIGN           >> ./out/${DESIGN}_map.sdc

# Write delay annotation file (to be used in ModelSim/other simulators)
write_sdf -design $DESIGN   >> ./out/${DESIGN}_map.sdf

# Generate overall synthesis metrics in HTML format
report_metric -format html -file synthesis_report.html
```

After synthesis, it generates all the outputs: the mapped netlist (.v), an updated constraint file (.sdc), a delay annotation file (.sdf), and an HTML synthesis summary.

```
# Generate timing report (with full pin names for clarity)
report timing  -full_pin_names   >> ./RPT/${DESIGN}_timing.rpt

# Generate area report (cell count, area usage, etc.)
report area                      >> ./RPT/${DESIGN}_area.rpt

# Generate gates report (number of gates, breakdown of logic types)
report gates                     >> ./RPT/${DESIGN}_gates.rpt

# Generate power report (power estimation from synthesized netlist)
report power                     >> ./RPT/${DESIGN}_power.rpt

# Generate Quality of Results (QoR) report
# Includes logic levels, fanout, etc.
report qor -levels_of_logic      >> ./RPT/${DESIGN}_qor.rpt
```

It also produces detailed reports on timing, area, gate count, power, and overall quality of results (QoR).

Changing Required in this script:

As this file was already provided with the paths defined, before running this file we had to change different paths according to our file location. First I changed the path of project environment file path, then the path of top.v and constraint.sdc. And after changing all the paths the script.tcl is run and following results are shown.

```
################################################################################
## Environment Setup
################################################################################

# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/Downloads/Sidra_PD_UART/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


################################################################################
## Design Setup
################################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       UART_Top

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

```
################################################################################
## Read and Elaborate Design

# IMPORTANT: Update the path if your Verilog source is located elsewhere

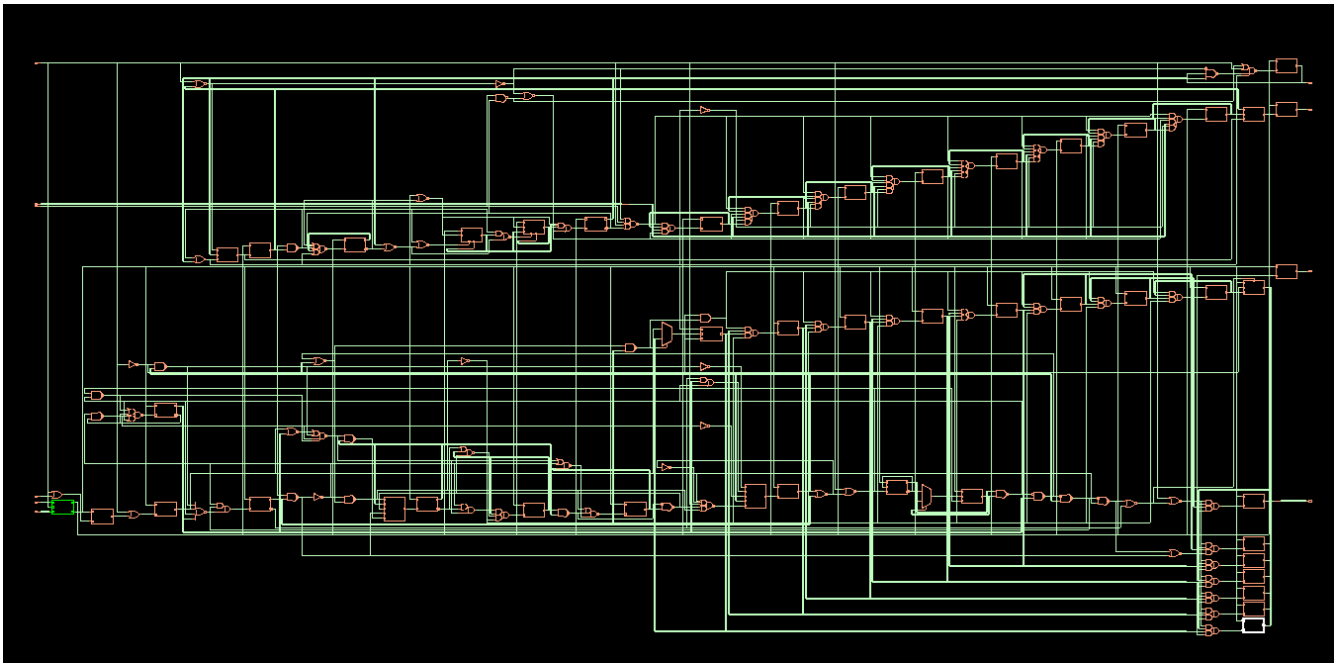
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/UART_Top.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Transmitter.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Receiver.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/baud_rate_generator.sv}
```

Explanation:

The script.tcl file is an automation script used to run the synthesis process for the processor design in the synthesis tool (such as Synopsys Design Compiler). It contains all the necessary commands to read the RTL files, define the working library, apply design constraints, and generate the synthesized gate-level netlist. The script usually starts by setting up the environment and specifying the target and link libraries, followed by reading the Processor Verilog files and the .sdc constraint file (which defines timing, clock, and input/output delays). After that, the script runs the synthesis command to map the RTL design onto standard cells, performs optimization for area and timing, and finally writes out reports for timing, area, and power, along with the synthesized netlist. This helps ensure a consistent, automated, and repeatable synthesis process without manually entering commands each time.

Synthesize:

Netlist:



This is netlist generated first time when we run the script.tcl, It shows how different gates, inputs and outputs are connected. It shows how different standard cells from library file are placed against RTL design. Different names are given like DFCN01, it specifies that it's a D flip flop. This netlist contains all the input and output connections, internal wires, and instantiated cells that make up the final synthesized design.

Files generated:

top_map.v(netlist)

top_gates.rpt

top_area.rpt

top_timing.rpt

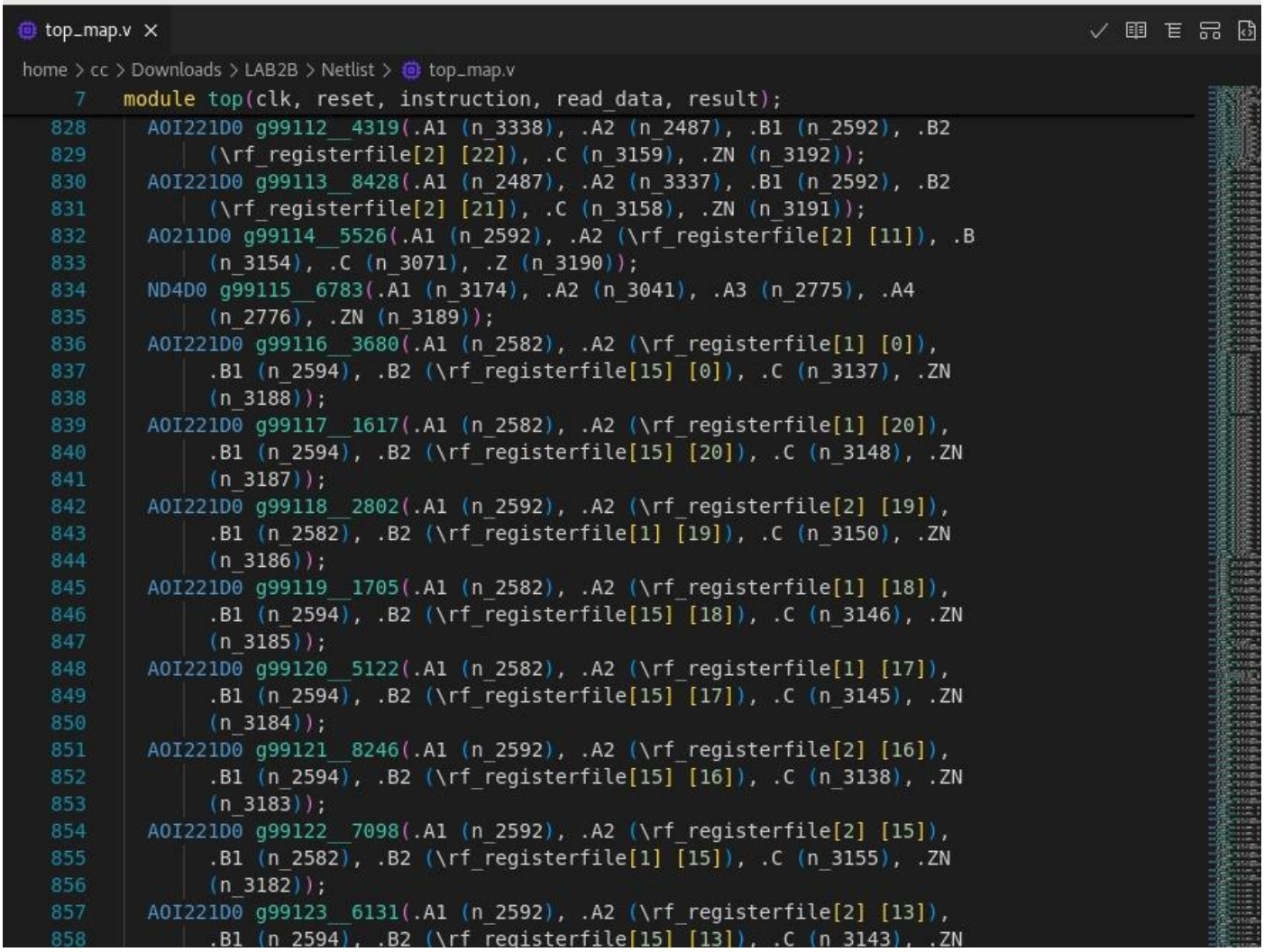
top_qor.rpt

top_power.rpt

top_map.sdc (in “out” subdirectory)

top _map.sdf (in “out” subdirectory)

top_map.v:



Explanation:

This part of the synthesized top_map.v file shows the gate-level version of my single-cycle processor frontend after synthesis. It means that my Verilog code has been converted into basic hardware components like AND, OR, NAND, NOR, and NOT gates. Each line represents how these gates are connected to form the logic of the processor, such as the register file and instruction decoding parts. Signals like n_2019 and \rf_registerfile[15][11] are internal connections and register bits used by the design. In simple terms, this file shows that my processor’s RTL design has been successfully converted into actual hardware logic that can be implemented on a chip.

Top gates Report:

```
NR2XD3            2     12.240    tcbn65lptc
NR3D0             4      8.640    tcbn65lptc
NR4D0             5     12.600    tcbn65lptc
OA211D0           1      2.880    tcbn65lptc
OA211D1           1      2.880    tcbn65lptc
OA21D0            9     22.680    tcbn65lptc
OA21D1           14     35.280    tcbn65lptc
OA221D0           4     14.400    tcbn65lptc
OA22D0            2      5.760    tcbn65lptc
OAI211D0         12     30.240    tcbn65lptc
OAI21D0          33     71.280    tcbn65lptc
OAI21D1          34     73.440    tcbn65lptc
OAI221D0         33     95.040    tcbn65lptc
OAI222D0          9     29.160    tcbn65lptc
OAI22D0          42    105.840    tcbn65lptc
OAI22D1           1      2.520    tcbn65lptc
OAI31D0           4     10.080    tcbn65lptc
OAI32D0           2      5.760    tcbn65lptc
OR2D0            11     23.760    tcbn65lptc
OR2XD1            8     17.280    tcbn65lptc
OR3D1             1      2.520    tcbn65lptc
OR4D0             7     20.160    tcbn65lptc
XNR2D1            3     10.800    tcbn65lptc
XOR3D1            1      6.120    tcbn65lptc
-----------------------------------------------
total          4759  16794.720
```

```
Type       Instances    Area   Area %
----------------------------------------
sequential          1057  7182.000   42.8
inverter             147   173.160    1.0
buffer                32    46.080    0.3
logic               3523  9393.480   55.9
physical_cells         0     0.000    0.0
----------------------------------------
total               4759 16794.720  100.0
```

Explanation:

This gate report shows the detailed result of the synthesis process for my single-cycle processor frontend. It lists all the logic gates and flip-flops that the synthesis tool (Cadence Genus) used to convert my Verilog code into actual hardware. Each type of gate, such as AND, OR, NAND, NOR, AOI (AND-OR-Invert), and OAI (OR-AND-Invert), is shown along with how many times it is used and how much area it occupies on the chip. The “sequential” part includes flip-flops and registers, which store data between clock cycles. The report also

shows that there are a total of 4614 cells used, taking a total chip area of about 16593 µm². Around 43% of the area is used by sequential elements (registers and flip-flops), and about 56% is used by logic gates that perform operations. In simple words, this report tells how my processor design has been physically built from basic digital building blocks and how much area each part of it occupies on the chip.

Top_area.rpt:

```
====================================================================
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 05 2025  05:59:30 pm
  Module:                 top
  Operating conditions:   NCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
====================================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area      Wireload
--------------------------------------------------------------------------------
top                   4759  16794.720     0.000    16794.720 ZeroWireload (S)
  (S) = wireload was automatically selected
```

Explanation:

This area report shows the total hardware size of my single-cycle processor frontend after synthesis. It tells that the design, named “top”, contains 4759 cells (logic gates and flip-flops) in total. The cell area is 16794.720 µm², which represents the total silicon area required to implement all the components. The net area is 0.000, meaning the wiring area was not separately calculated because a “zero wireload” model was used — this means the tool assumed ideal wire connections for simplicity. In short, this report confirms that my processor design is compact and occupies a total chip area of 16794.720 µm², based only on logic cell usage.

top_timing.rpt:

```
Path 1: MET (0 ps) Late External Delay Assertion at pin result[5]
           Group: clk
     Startpoint: (F) instruction[0]
           Clock: (R) clk
        Endpoint: (F) result[5]
           Clock: (R) clk

                                Capture       Launch
          Clock Edge:+             4200            0
          Drv Adjust:+                0            0
         Src Latency:+                0            0
         Net Latency:+                0 (I)        0 (I)
               Arrival:=          4200            0

        Output Delay:-              1000
         Uncertainty:-              2000
     Required Time:=                1200
      Launch Clock:-                   0
       Input Delay:-                1000
         Data Path:-                 200
             Slack:=                   0
```

Explanation:
This timing report shows the setup timing check for the single-cycle processor frontend. The analysis is for the path that starts from the instruction input and ends at the result output. The total data path delay is 200 ps, and after considering the input delay (1000 ps), output delay (1000 ps), and uncertainty (2000 ps), the required time is 1200 ps.
Since the slack is 0 ps, it means the design exactly meets the timing requirement — there are no timing violations, but also no extra margin left. In simple terms, the circuit runs right at the limit of the clock timing but still performs correctly without any delay issues.

top_qor.rpt:

```
Clock Period
-------------
clk   4200.0


  Cost    Critical         No of gates on  Violating
 Group   Path Slack  TNS   Critical Path    Paths
-----------------------------------------------------
clk             0.0   0.0              6          0
default    No paths   0.0
-----------------------------------------------------
Total                 0.0                         0

Instance Count
--------------
Leaf Instance Count             4759
Physical Instance count            0
Sequential Instance Count       1057
Combinational Instance Count    3702
Hierarchical Instance Count        0

Area
----
Cell Area                          16794.720
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    16794.720
Net Area                           0.000
Total Area (Cell+Physical+Net)     16794.720

Max Fanout                         1024 (clk)
Min Fanout                         1 (ALUResult[0])
Average Fanout                     3.1
Terms to net ratio                 4.1281
Terms to instance ratio            4.2383
Runtime                            58.69125799999999 seconds
Elapsed Runtime                    58 seconds
Genus peak memory usage            1960.52
Innovus peak memory usage          no_value
Hostname                           ncdc-0053
```

Explanation:

This report summarizes how the synthesized single cycle performed. The tool used a clock period of 4200 ps and the design has a positive slack of 0 ps on the critical clock path — so it meets timing with a small margin. The critical path spans 12 gates, and there are no timing violations (TNS = 0). The netlist contains 4759 leaf instances (1057 sequential registers and 3702 combinational cells) and a total cell area of 16794.720 µm². The

clock net has the highest fanout (1024) while the average fanout is 3.1, which indicates reasonable net loading overall.

top_power.rpt:

```
Instance: /top
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
  Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
    memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  register     3.28131e-07  2.74416e-03  1.17825e-04  2.86231e-03  75.35%
    latch      5.81704e-09  2.38083e-05  5.54467e-06  2.93588e-05   0.77%
    logic      2.94966e-07  5.30902e-04  3.75585e-04  9.06781e-04  23.87%
     bbox      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
    clock      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
      pad      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       pm      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
  Subtotal     6.28915e-07  3.29887e-03  4.98954e-04  3.79845e-03  99.99%
Percentage           0.02%       86.85%       13.14%      100.00% 100.00%
  ------------------------------------------------------------------------
```

This power table is for the /top module and shows how much power each part of the design uses. The total power is ≈1.564 mW (1.56441×10^{-3} W). That breaks down into leakage (very small, 0.000000619 W), internal power (≈1.368 mW, 87.5%) and switching power (≈0.196 mW, 12.5%). Most of the power (about 76.4%) is consumed by registers (flip-flops and storage) because they toggle and charge internal nodes during clocked operation. Combinational logic uses about 22.8%, and latches are negligible (~0.8%). The clock, pads and memories show no separate power here. In simple terms: the design's dynamic activity (mainly inside registers) is the main source of power — total is low (~1.6 mW) and dominated by internal register activity.

top_map.sdc(out directory):

```
set sdc_version 2.0

set_units -capacitance 1000fF
set_units -time 1000ps

# Set the current design
current_design top

create_clock -name "clk" -period 4.2 -waveform {0.0 2.1} [get_ports clk]
set_clock_transition 0.1 [get_clocks clk]
set_clock_gating_check -setup 0.0
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports reset]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[31]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[30]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[29]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[28]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {instruction[27]}]
```

```
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {result[0]}]
set_wire_load_mode "segmented"
set_clock_uncertainty -setup 2.0 [get_clocks clk]
set_clock_uncertainty -hold 1.0 [get_clocks clk]
set_clock_uncertainty -setup 0.01 [get_ports clk]
set_clock_uncertainty -hold 0.01 [get_ports clk]
```

This file is generated by cadence tool genus. It has the timing and design constraints that define how the synthesized design should behave in terms of clock, input and output. It tells how much margin should be used in timing analysis. It defines a 10ns clock period for the clock with 0.1ns of rise/fall transition and sets input and output delay constraints of 1ns for reset and count signals.

top_map.sdf:

```
  (CELL
     (CELLTYPE "INVD1")
     (INSTANCE g41878)
     (DELAY
        (ABSOLUTE
          (PORT I (::0.0))
          (IOPATH I ZN (::278) (::251))
        )
     )
  )
  (CELL
     (CELLTYPE "CKND2D1")
     (INSTANCE g98997__2398)
     (DELAY
        (ABSOLUTE
          (PORT A1 (::0.0))
          (PORT A2 (::0.0))
          (IOPATH A1 ZN (::154) (::144))
          (IOPATH A2 ZN (::155) (::133))
        )
     )
  )
```

Explanation:

This file contains the timing information of each cell. It has the information generated after synthesis for the counter circuit. It has the actual delays and setup, hold, recovery and removal times for every cell in the design.

Explanation of top.sdc

An SDC (Synopsys Design Constraints) file tells the synthesis and timing analysis tools (like Cadence Genus) about the timing environment of design. It does not describe the logic itself, but the constraints under which the logic should be optimized and verified.

Explanation:

In real hardware, clock is not perfect stable, clock. There are small variations called clock uncertainties when the clock edge arrives at different flip flops. These uncertainties are based on skew and jitter.
Jitter is the variations of clock edge time, due to oscillator noise while the skew is difference in clock edge at different points due to routing delays. In this sdc file, For **setup analysis**, we subtract **2 ns** from the available time. This means the tool assumes the data must arrive **2 ns earlier**, ensuring that even if the clock edge jitters forward, the design still works. For **hold analysis**, we add **1 ns** margin. This prevents data from arriving too early if the clock edge shifts backward. At the chip's clock input pin, assume the clock is almost perfect, with only 0.01 ns of variation.

Gate Level Simulation (Post Synthesis):

Gate level simulation is done after synthesis to check that circuit will work once it is converted from RTL into logic gates. It helps us to verify that the functionality hasn't changed and that delays from real gates and wires don't cause issues.

Files needed for gate level Simulation:

1. **Synthesized netlist:**

It is the top_map.v file which is Verilog code after synthesis. In this file design is converted into actual standard cells.

2. **Testbench file:**

This is tb_top.v file it is the testbench same as we used before synthesis to apply inputs and output checks.

3. **SDF file:**

This is top_map.sdf file whoch contains actual delay values of every gate and wire. It is generated by genus.

4. **Technology library file:**

This file is tcbn651lp.v, and it has the Verilog modules of the standard cells used in the netlist.

Changing SDF Annotation:

This tells the simulator to use the real gate delays from synthesis.

Running the Simulation:

Csh
It opens the C shell.
Source cshrc
It sets up environment variable and tools.

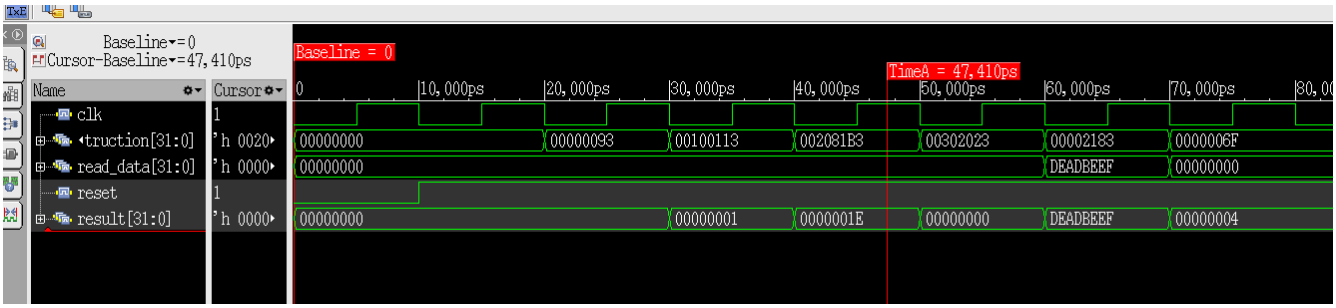
xrun -timescale 1ns/10ps top_netlist_dft.v tb_top.sv -v tcbn65lp.v -access +rwc -define SDF_TEST -mess -gui

Design summary:

```
                streams:   0, words:     0
Building instance specific data structures.
Loading native compiled code:     ..................... Done
Design hierarchy summary:
                          Instances  Unique
        Modules:               4761     105
        UDPs:                  5348       4
        Primitives:           15429       9
        Timing outputs:        4820     107
        Registers:             1062      11
        Scalar wires:          5880       -
        Expanded wires:          96       3
        Initial blocks:           3       3
        Pseudo assignments:       5       -
        Timing checks:        10983       -
        Simulation timescale:   10ps
Writing initial simulation snapshot: worklib.top_tb:sv
```

Explanation:
This design hierarchy symmary is shown after running GLS. It shows the structural breakdown of my simulated design. It tells how many modules, primitives and componenets it contains after synthesis.

Cadence Waveform:



Command:

genus -f DFT_script.tcl

Synthesis with DFT:

I am asked to do synthesis with DFT because without it, I cannot properly test or verify my Processor design after the chip is fabricated. In a normal synthesis flow, I only get a working circuit, but I cannot see what happens inside each flip-flop or detect any internal faults once the chip is built. With DFT, the tool

automatically inserts special scan cells and connects them into scan chains. These allow me to shift test data into the design and observe the internal outputs directly during testing. This means I can check whether every part of my design is working correctly or if any manufacturing defect exists. So, DFT doesn't change the actual functionality of my design it just makes it testable and ensures that after fabrication, I can easily verify and debug my chip.

Results:

```
  Checking DFT rules for 'top' module under 'muxed_scan' style

Detected 0 DFT rule violation(s)
        Summary of check_dft_rules
        **************************
        Number of usable scan cells: 124
Clock Rule Violations:
----------------------
          Internally driven clock net: 0
              Tied constant clock net: 0
                   Undriven clock net: 0
        Conflicting async & clock net: 0
                      Misc. clock net: 0

Async. set/reset Rule Violations:
---------------------------------
        Internally driven async net: 0
              Tied active async net: 0
                 Undriven async net: 0
                    Misc. async net: 0

   Total number of DFT violations: 0

  Total number of Test Clock Domains: 1
  Number of user specified non-Scan registers:   0
      Number of registers that fail DFT rules:   0
      Number of registers that pass DFT rules: 1024
  Percentage of total registers that are scannable: 100%
```

Explanation:

There is no violation observed.
During scan insertion, DFT tools need all flip-flops to be controlled by a recognized clock so test patterns can shift and capture data correctly.

Slacks Information:

```
Incremental optimization status
================================
                                  Group
                                 Tot Wrst     Total - - DRC Totals - -
                          Total  Weighted      Neg      Max       Max
Operation                  Area   Slacks      Slack    Trans      Cap
 init_delay               18668        0         0         0        0
```

Scanning Flip Flops:

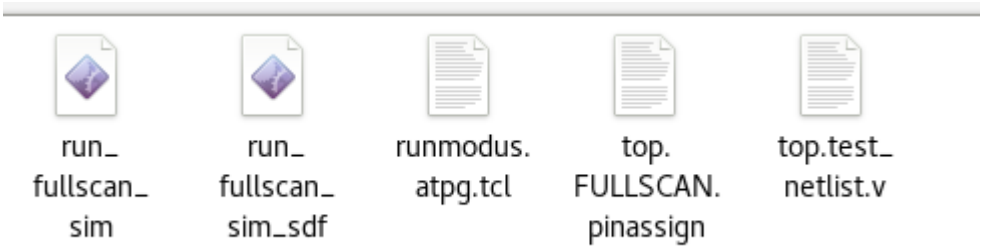
```
@file(DFT_script.tcl) 131: write_hdl > outputs/top_netlist_dft.v
@file(DFT_script.tcl) 134: write_sdc > outputs/top_sdc_dft.sdc
Finished SDC export (command execution time mm:ss (real) = 00:00).
@file(DFT_script.tcl) 137: write_sdf -nonegchecks -edges check_edge -timescale ns -recrem split -setuphold split > outputs/dft_delays.sdf
@file(DFT_script.tcl) 140: write_scandef > outputs/top_scanDEF.scandef
@file(DFT_script.tcl) 148: write_sdc $DESIGN >> ./out/${DESIGN}_map.sdc
Finished SDC export (command execution time mm:ss (real) = 00:00).
@file(DFT_script.tcl) 151: write_sdf -design $DESIGN >> ./out/${DESIGN}_map.sdf
```

```
Info    : Done incrementally optimizing. [SYNTH-8]
        : Done incrementally optimizing 'top'.
      flow.cputime  flow.realtime  timing.setup.tns  timing.setup.wns  snapshot
UM:*                                                                 syn_opt_incr
@file(DFT_script.tcl) 122: write_hdl $DESIGN -mapped >> ./Netlist/${DESIGN}_map.v
@file(DFT_script.tcl) 125: report_scan_chains
Reporting 1 scan chain (muxed_scan)

Chain 1: top_chain
  scan_in:      scan_in
  scan_out:     scan_out
  shift_enable: SE (active high)
  clock domain: clk (edge: rise)
```

Files:

These are the repots generated after the synthesis with scan flip flops. Scan chain report tells how many chains, how many FFs are inserted. Timing report shows the slack informations (after scan insertion). Power/area report tells how much overhead DFT added.



Xcelium Simulation Script (FULLSCAN run)
This shell script runs the Cadence Xcelium simulator to verify my Processor design after DFT (Design for Test) insertion. It uses test vectors generated by Modus ATPG to check if the scan chain and logic are working correctly. It loads the gate-level netlist, standard cell library, and generated testbench files. The main goal is to make sure that the inserted scan chain and test patterns work as expected in simulation.

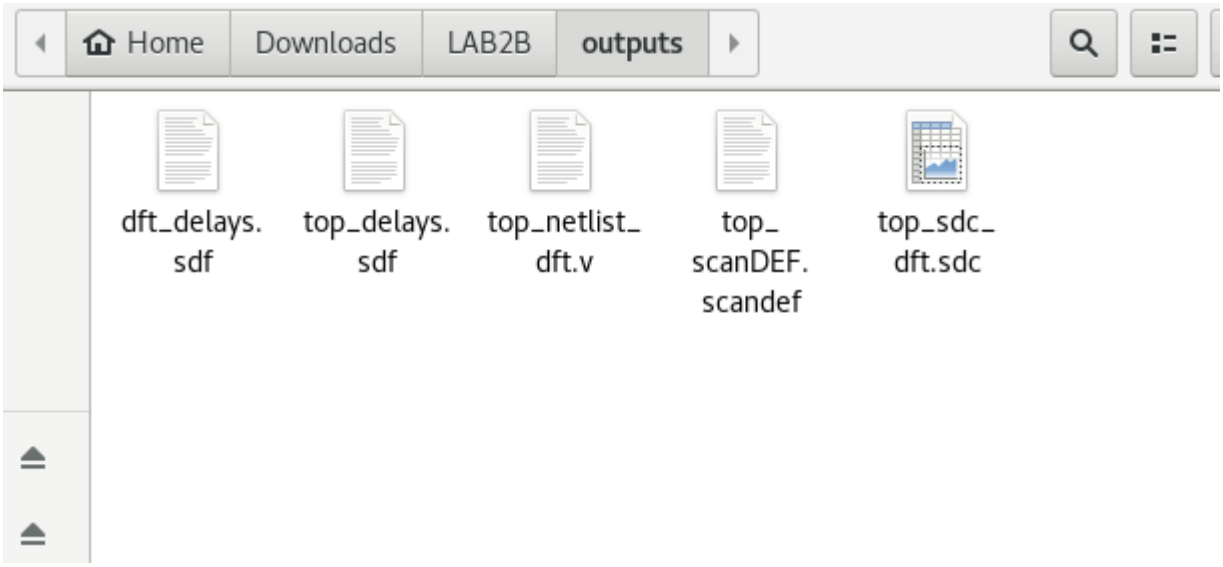
Xcelium Simulation Script (FULLSCAN TIMED run)
This script is similar to the first one but includes timing simulation using SDF (Standard Delay Format) files. It tests how the design behaves with real timing delays after synthesis or DFT insertion. By adding the +sdf_file and +TIMING options, this simulation shows real hardware timing behavior and ensures that the design still functions correctly under actual delay conditions.

Modus ATPG Script
This Tcl script runs the Modus ATPG tool to automatically generate test patterns for my synthesized processor design. It builds the logic model, test model, and fault model, then creates scan chains and test vectors to detect possible manufacturing faults. It reports test coverage and stores all generated results. Basically, it makes the design test-ready by generating the data needed for DFT verification.

Pin Assignment File (.pinassign)
This file defines how the test pins are assigned for DFT. For example, it maps reset to test mode, SE to scan enable, clk as test clock, and scan_in / scan_out as scan data pins. These mappings help Modus and Xcelium know which pins are used for test control and scan operations during simulation and ATPG.

Top.test_netlist.v (Gate-Level Netlist)
This is the final synthesized netlist of my processor design after DFT insertion. It includes all standard cells like NANDs, flip-flops, and multiplexers from the 65nm library. I can see how each flip-flop is connected with scan input, scan enable, and scan output. This file shows how my RTL design is transformed into an actual hardware-level representation that can be implemented on silicon and tested with scan chains.

Now, go to the output folder in your directory.



Top.test_netlist.v (Gate-Level Netlist File)
This file represents the final synthesized netlist of my Processor design after DFT insertion. It shows the complete hardware-level structure where all Verilog behavioral code is converted into standard cells like flip-flops, gates, and multiplexers from the 65nm library. It also includes scan-related pins such as scan_in, scan_out, and SE to make the design testable using scan chains. Each module like tb_top and Top is defined at the gate level, showing how data flows through logic cells during operation and testing. This file is mainly used for gate-level simulation, static timing analysis, and test verification after synthesis.

SCAN CHAIN (.scan) File
This file lists the scan chain configuration created during the DFT process. It defines how flip-flops are connected serially from scan_in to scan_out to form scan chains. Each flip-flop entry includes its scan input, output, and the clock domain it belongs to. The purpose of this file is to make sure all sequential elements can be accessed and tested during manufacturing. It also helps the ATPG tool generate and apply test patterns to verify that the chip works correctly after fabrication.

SDC (Timing Constraint) File
This file contains all the timing constraints for my Processor design during synthesis and optimization. It includes commands like create_clock to define the main system clock period (10ns), and sets input/output delays, clock transition, and clock uncertainty. These constraints guide the synthesis tool to achieve timing closure by ensuring all paths meet setup and hold requirements. It also helps determine the maximum operating frequency of my design by adjusting the clock period or timing constraints.

Post Synthesis of Code with DFT:

```
module top_tb;


logic clk;
logic reset;
logic [31:0]instruction,read_data,result;
//logic SE,scan_in,scan_out;

top dut (
    .clk(clk),
    .reset(reset),
    .instruction(instruction),
    .read_data(read_data),
    .result(result)
 .SE(SE),
  .scan_in(scan_in),
   .scan_out(scan_out)
);


initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

`ifdef SDF_TEST
initial begin
    $sdf_annotate("/home/cc/Downloads/LAB2B/RTL/top_map.sdf", top_tb.dut, "sdf.log", "MAXIM
end
`endif


initial begin
```

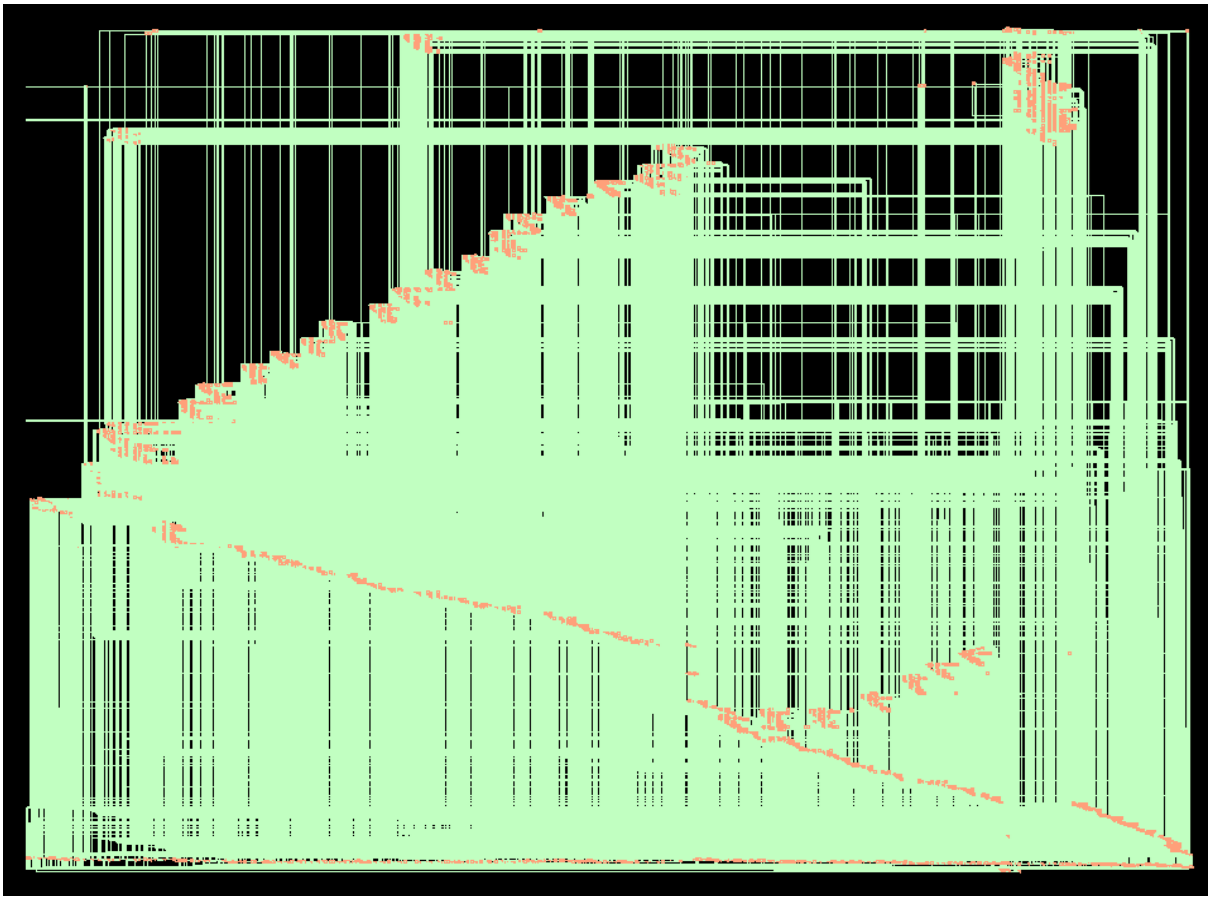
Command:

xrun -timescale 1ns/10ps top_netlist_dft.v tb_top.sv -v tcbn65lp.v -access +rwc -define SDF_TEST -mess -gui

Terminal output:

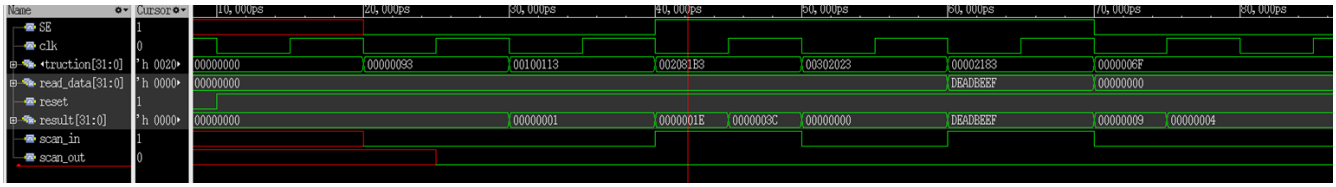
```
  Building instance specific data structures.
  Loading native compiled code:     ..................... Done
  Design hierarchy summary:
                             Instances  Unique
          Modules:                4811     104
          UDPs:                  20612       4
          Primitives:            34766       9
          Timing outputs:         4870     106
          Registers:              1065      13
          Scalar wires:           5934       -
          Expanded wires:           96       3
          Initial blocks:            3       3
          Pseudo assignments:        8       -
          Timing checks:         55527       -
          Simulation timescale:   10ps
  Writing initial simulation snapshot: worklib.top_tb:sv
std          /STD
```

Schematic view:



Explanation:
This netlist now have the inputs of scan enables, scan_in as inputs and scan_out.

Waveform:



Explanation:

In my waveform, At the start, the clock keeps toggling every 5 ns, and reset is applied briefly to initialize all signals. After reset, the test begins by sending a sequence of RISC-V instructions to the processor. Each instruction (like addi, add, sw, lw, and jal) executes in a new cycle, and the waveform shows changes in instruction, read_data, and result signals accordingly.
The scan signals (SE, scan_in, scan_out) are used to test the internal flip-flops. When SE = 1, the circuit enters scan mode, and data shifts through the scan chain — visible in the waveform as serial toggling of scan_in and corresponding output at scan_out. When SE = 0, the design runs in normal functional mode, executing the given instructions.
In simple words, the waveform confirms both functional behavior (processor executing instructions) and DFT operation (scan chain working correctly).

Critical Analysis:

In this lab, I designed and integrated different modules to form a complete single-cycle processor. Each module including the Program Counter, Instruction Memory, Control Unit, Register File, and ALU was developed and verified individually before integration. The design successfully executed basic instructions within one clock cycle, demonstrating the fundamental working of a processor datapath. During implementation, the main challenge was ensuring proper signal connectivity and timing synchronization between modules, especially while integrating the instruction memory and control unit. Debugging the control signals and verifying instruction decoding required careful observation of simulation waveforms. Overall, the lab provided a clear understanding of how data flows through the processor and how control signals coordinate the execution process. However, the single-cycle design is not efficient for complex instructions, as all operations complete in one long cycle. Future improvement could involve implementing a pipelined design to enhance performance and clock efficiency.